FORWARD DEPLOYED ENGINEERING

LEC-1:

1. Forward Deployed Engineer (FDE)

An **FDE** is a software engineer who works **directly with customers** to understand their problems and build real-world software/AI solutions.

FDE Workflow

1. Customer Problem
2. Understand Problem
3. Investigate Data & Systems
4. Design Solution
5. Build Software/AI
6. Connect APIs/Databases
7. Deploy
8. Test & Improve

FDE vs Software Engineer

Software Engineer	**FDE**
Builds product/features	Solves customer problems
Works mainly with engineering team	Works directly with customers
Usually receives requirements	Often discovers requirements
Focuses on reusable products	May build customized solutions
Coding-focused	Coding + business + communication

FDE Mindset

Software Engineer: "What should I build?"
FDE: "What problem does the customer have, and how can I solve it?"

---

2. Agent Harness

An **agent harness** is the software around an AI model that **helps the AI work as an agent**.

Simple idea

LLM = Brain 🧠
Harness = Tools and environment that allow the brain to work

Harness includes

- Instructions
- Memory/context
- Tools
- Permissions
- State
- Tool results
- Error handling
- Execution control

Example: Coding Agent

You say:

"Fix the bug in my React app."

The **LLM** decides how to solve the problem.

The **harness**:

1. Give access to project
2. Read files
3. Run commands
4. Receive results
5. Modify code
6. Test
7. Repeat if necessary

Key Definition

Agent Harness = Runtime/orchestration layer that turns an LLM into a useful tool-using agent.

---

3. FDE + Agent Harness

An FDE can use an **agent harness** to build and deploy AI solutions faster.

Customer Problem

1. FDE
2. AI Agent + Harness
3. Tools + Data + APIs
4. Customer Solution

FORWARD DEPLOYED ENGINEERING

In **Claude Code**, **connectors** and **MCP** are ways to let Claude interact with external systems and tools.

1. MCP — Model Context Protocol

MCP is a standard way for an AI model to connect to external tools and data.

Think of it as a **bridge**:

Claude Code

↓

MCP

↓

┌──────────┬──────────┬──────────┐

GitHub Database Slack

Instead of Claude only reading your local code, MCP can give it access to things like:

- GitHub
- Databases
- Slack
- Jira
- APIs
- Internal company systems

Example

You ask:

"Check the Jira tickets related to this bug and fix the issue."

Claude Code can use an MCP server to:

Claude

↓

MCP

↓

Jira → Get ticket

↓

Claude understands problem

FORWARD DEPLOYED ENGINEERING

↓

Local code → Modify code

↓

Tests

---

2. Connector

A **connector** is essentially a **connection between Claude and an external service/system**.

For example:

Claude Code

|

├── GitHub Connector → Repositories

├── Slack Connector → Messages

├── Jira Connector → Tickets

└── Database → Business data

A connector may use **MCP** underneath to provide that connection.

Easy way to remember

MCP = the protocol/standard
Connector = the connection to a specific service

Think of:

USB = protocol/standard
USB cable/device = connection

---

3. Why this matters for an FDE

This is especially important for a **Forward Deployed Engineer**.

Imagine a customer has:

Salesforce

|

Jira

|

FORWARD DEPLOYED ENGINEERING

PostgreSQL

|

GitHub

|

Slack

An FDE can build an AI agent that connects these systems through tools/MCP.

Then the agent can potentially:

Understand the customer's problem → retrieve company data → use tools → make changes → verify the result.

That's a major part of what makes modern AI agents useful in real enterprise environments.

Sure. In the same architecture, **SoR (System of Record)** and **Agent Factory** are two different concepts.

1. SoR — System of Record

A **System of Record** is the **official/authoritative place where business data is stored**.

Think:

SoR = Source of Truth

For example, a company might have:

Salesforce → Customer records

PostgreSQL → Orders / transactions

SAP → Finance / ERP data

Jira → Projects / tickets

HR System → Employee records

If an AI agent needs to know:

"How much did Customer X purchase?"

It should retrieve the information from the appropriate **System of Record**, rather than simply trusting its own memory.

Example

AI Agent

↓

MCP / Connector

↓

PostgreSQL

↓

Official customer data

So:

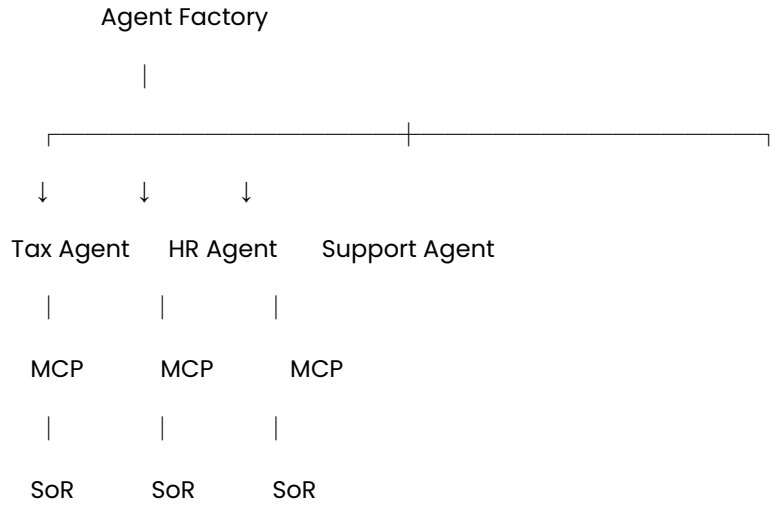
MCP = how the agent connects
SoR = where the trusted data lives

---

2. Agent Factory

An **Agent Factory** is a system/platform for **creating and managing AI agents**.

Instead of building every agent completely from scratch, the factory provides reusable components.



For example, an **Agent Factory** might provide:

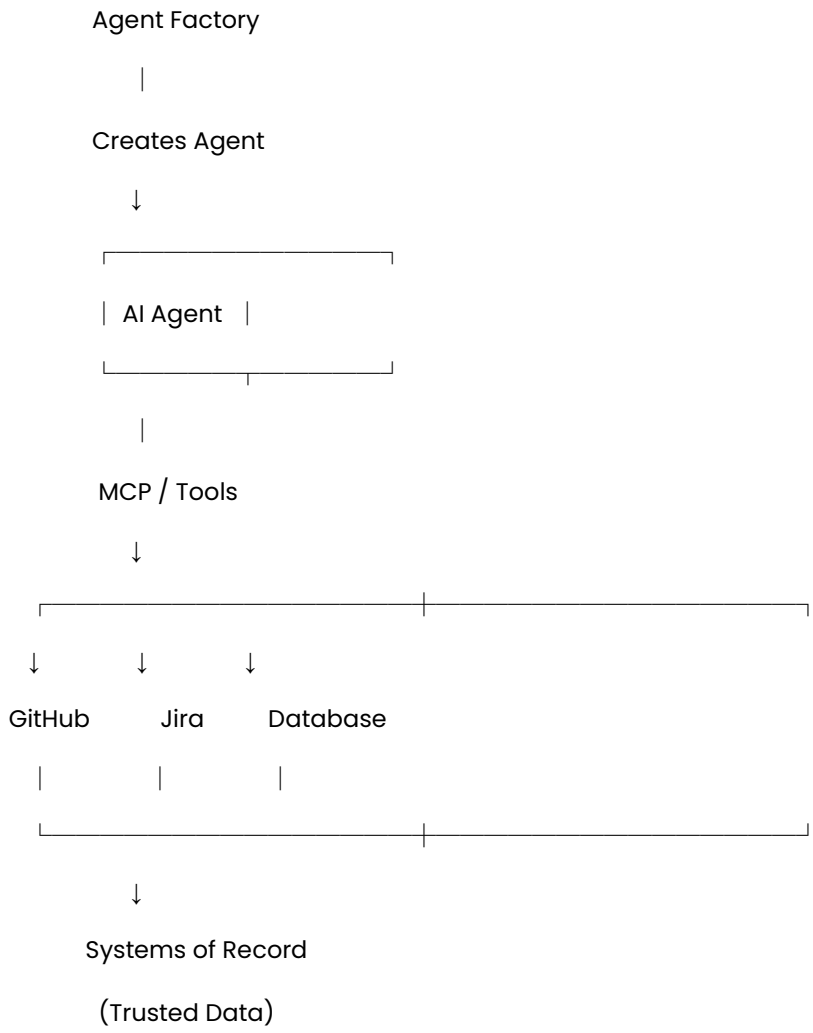
- Agent templates
- Instructions/prompts
- Tools
- MCP connections
- Memory
- Permissions
- Security

- Evaluation/testing
- Deployment
- Monitoring

---

3. Put Everything Together

Now you can see how the pieces relate:



Easy way to remember

Concept	Simple meaning
Agent Factory	🏭 Builds/manages agents
Agent	🤖 Performs a task

FORWARD DEPLOYED ENGINEERING

Concept	Simple meaning
MCP	🧰 Standard bridge to tools/data
Connector	🔌 Connection to a specific service
SoR	🗄️ Official source of business data
Harness	⚙️ Runtime that controls the agent

In an FDE scenario

An FDE might be asked:

"Build an AI tax agent for an automotive company."

The FDE could use an **Agent Factory** to create the agent, a **harness** to run it, **MCP/connectors** to access company systems, and the company's **Systems of Record** to obtain authoritative financial/tax data.

That gives you the bigger picture:

FDE → builds the solution
Agent Factory → creates/manages agents
Harness → runs/controls agents
MCP/Connectors → give agents access to systems
SoR → provides trusted business data

FORWARD DEPLOYED ENGINEERING

LEC-2:

10–80–10 Rule

The **10–80–10 rule** describes how to work with a general AI coding agent like Claude Code.

1. First 10% — Intent & Planning

- Define **what you want to build**.
- Clearly understand the problem and requirements.
- Decide the expected outcome.

2. Middle 80% — Execution

- The **general agent** does the mechanical work.
- It handles coding, implementation, file changes, testing, etc.
- Example: **Claude Code** performs most of the development work.

3. Final 10% — Verification & Ownership

- You **review and verify** the agent's work.
- Check whether it actually meets your requirements.
- Test the result and fix anything incorrect.
- **You remain responsible for the final outcome.**

Simple Flow

10% 80% 10%

Intent AI Execution Verification

Planning → Coding/Work → Review/Test

Key idea:

You define the intent → AI does the mechanical work → You verify and own the result.

FORWARD DEPLOYED ENGINEERING

FDE 80/20 Rule & Platform — Simple Notes

1. What is the 80/20 idea?

An FDE should **not build everything from scratch** for every customer.

Instead:

80% = Already-built platform

20% = Customer-specific customization

The FDE takes the existing platform and adapts it to the customer's needs.

---

2. Example

Suppose you build a **Car Leasing Platform**.

The existing 80% already contains:

- Customer management
- Vehicle management
- Payments
- Contracts
- Reports
- Authentication
- Basic workflows

A new customer needs:

"We have a special leasing approval process."

The FDE builds the **20% customization** for that customer.

---

3. The Big Problem Before AI

Imagine you have **50 customers**.

If you customize the platform separately for every customer:

Customer 1 → Version A

Customer 2 → Version B

Customer 3 → Version C

...

Customer 50 → Version Z

Now you have **50 different versions to maintain**.

This creates huge problems:

- Difficult maintenance
- Bug fixes must be repeated
- Updates become complicated
- Features become inconsistent
- Developers don't know which version to change

---

4. The Solution: Merge Good Ideas Back

When an FDE creates something useful for one customer:

Customer customization

↓

Useful new feature

↓

Merge into main platform

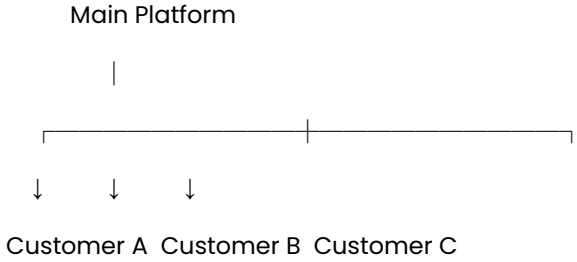
↓

All customers can receive it

So instead of:

50 separate versions ❌

you aim for:



(small changes) (small changes) (small changes)

---

5. Why AI Changes Things

With AI coding agents such as **Claude Code**, an FDE can potentially:

- Understand an existing large codebase
- Implement customer-specific changes
- Identify reusable improvements
- Refactor code
- Create tests
- Merge useful changes back into the main platform

This makes it much easier to maintain the **common 80%** while keeping customer-specific customization closer to **20%**.

Key idea

Build once → customize for customers → discover improvements → merge reusable improvements back into the platform → keep customer-specific differences small.

This is one of the important ideas behind **modern FDE work: you're not just building for one customer; you're continuously improving the platform while solving customer problems.**

FORWARD DEPLOYED ENGINEERING

LEC-03

Study Notes: KSOR & Foundations for AI Agents

Recap: "System of record" (traditional)

Question: what is a system of record?

Answer: it's the **source of truth** for a business process/state.

Examples:

- ERP for operations
- Accounting tools (e.g., QuickBooks)
- Databases storing business data (budgets, transactions, totals)

Key idea: these systems mainly define the **current state** with stored data. They usually *don't* include the human operational knowledge, policies, and decision logic needed to use the system correctly.

Before AI agents: knowledge is in humans

In organizations, humans run the "system of record." For example, an accountant knows how to:

- Use QuickBooks
- Apply the company's accounting rules
- Enter invoices correctly
- Follow compliance requirements

That operational knowledge is "in the human" and grows with experience.

AI agents are meant to take over human roles—so they need the same kind of knowledge. Traditional systems of record are not enough: they may store data/state, but they lack governed, trusted knowledge and rules.

Solution: KSOR (Knowledge System of Record)

Governing the authoritative knowledge layer for both humans and AI agents.

It provides a trusted, consistent "source of truth" knowledge layer that agents can use reliably in real workflows.

Access KSOR

Participants are directed to GitHub:

Repository: github.com/pavaversity/KSOR

FORWARD DEPLOYED ENGINEERING

Core explanation (medical example)

- **Question:** "What's a safe medicine dosage for a six-year-old?"
- Different AI helpers give different answers (A, B, C).
- **Problem:** trust—what's actually safe/correct?

Why answers differ

- **Different retrieval sources**: one helper pulls outdated info, another uses a different document, another relies on pretraining.
- **LLMs are stochastic**: even the "same" question can yield different outputs.

So helpers may not follow the same authoritative "rule version."

KSOR's approach: 8 core concepts

Concept 1: One official book

Use a single authoritative "official book" for the domain/workflow instead of many blogs/pages/sources.

Example: a verified medical reference provides safe dosage rules (with versioning). Implementation note: official docs live in a "knowledge folder" in the repo.

Concept 2: Stable vs fast-changing content

The official book should include slow-changing governance info, like:

- How to treat a fever
- Who can approve/sign
- Safe dosage rules

It should avoid frequently changing operational data (e.g., available beds, room assignments, remaining inventory).

Concept 3: Validation "stamp"

Each page needs an approval status ("stamp"). With stamp = trusted; without stamp = not trusted. Example elements include author/committee, date/version, "older versions are outdated," and approval.

Concept 4: Access control ("door"/permissions)

KSOR restricts who (humans or agents) can see/use which pages. Example: nurses can access dosage rules; visitors can't access restricted patient-room notes.

Concept 5: "I don't know"

If the answer isn't in the official book, the agent should refuse rather than guess. This avoids hallucinations and invented answers.

Concept 6: Show the page / cite references

When answering, the agent references the exact page and version it used, enabling verification.

Concept 7: Adding pages (approval workflow)

KSOR defines how knowledge gets added:

- Authorized author proposes a new page
- Committees review/approve
- Unapproved pages aren't accessible
- Approved pages become available to the relevant people/agents

Concept 8: Multiple access "doors"

KSOR can be consumed via multiple interfaces (four listed):

- **Website** (for humans)
- **List file** (an input surface for AI systems to discover what exists)
- **MCP connector** (a standard connector for agent retrieval)
- **OKF bundle** (a packaged deployable setup)

What changes (and what doesn't) with KSOR

- **Can change:** how explanations are personalized when connected via MCP.
- **Should not change:** the governed knowledge content and rules in the official book.

Demo logic: same question → grounded answers

- All helpers retrieve the same dosage rule from the official book (same page/version).
- Answers become consistent and validated.
- AI can personalize explanation, but the governed factual basis stays the same.

Reframing KSOR (less technical)

- A "system of record" isn't new; it's a known concept.
- Traditional record systems hold data/state, not human operational knowledge.
- Before AI, humans learned and applied knowledge by reading books/procedures.

Extended examples

FORWARD DEPLOYED ENGINEERING

Education example (university)

- **Enrollment system of record**: admissions, programs, courses, grades, etc.
- **Textbooks/knowledge**: what students learn; reference persists after graduation.

AI shifts learning because models are pretrained on books, but pretrained knowledge can be outdated or inconsistent.

"AI study mode" behavior

Different students can get different learning paths because LLMs are stochastic. For a continuous learning journey, you need:

- Consistent curriculum progression
- A learning record (what's been covered)

Do we need textbooks if AI was trained on books?

- **Against:** if a model's "knowledge cutoff" still works, it might answer without new materials.
- **For:** answers can be inconsistent; learning needs structured progression and tracking; AI answers can change each time.
- **Also:** web-based answers may be unreliable and should be verified against an authoritative source.

Conclusion: for complete journeys and deterministic progression, textbooks (or equivalent governed sources) are needed.

Four "why this breaks" reasons (later expanded)

1. **Non-determinism**: agent answers can change.
2. **Wrong data / wrong training**: incorrect info may be present.
3. **Hallucination**: the model may still guess wrong output.
4. **No access to private company data**: internal knowledge isn't on the public internet.

KSOR in education = the "textbook"

Just like textbooks keep student learning honest, AI agents need a governed "knowledge system of record."

Two systems of record for online education

- **KSOR (knowledge)**: the authoritative content ("textbook").
- **Learning/Student system of record**: tracks progress (admissions, courses, progression, struggles).

FORWARD DEPLOYED ENGINEERING

Enterprise shift: agent "digital twins" need consistency

Without a knowledge system of record, an expert-based bot may pull scattered, conflicting information and deliver inconsistent answers. KSOR keeps responses within expert boundaries and verified rules.

Clarification: KSOR reduces issues, it doesn't make LLMs perfectly deterministic

Attaching a full knowledge book doesn't remove stochasticity. But KSOR grounds answers in governed boundaries, reducing hallucination and pulling the correct source when in scope.

KSOR as a platform + vertical systems

KSOR is framed as a platform for building domain-specific "vertical" knowledge systems, like sales/accounting, education, or medical/policy-heavy domains. Students can also build their own or start from an "agent factory" system (beta 1).

RAG vs KSOR

- **RAG**: retrieves data semantically (e.g., search/vector retrieval).
- **KSOR**: governed authoritative knowledge layer with validation stamps, versioning, access control, and trusted sources.
- **RAG** can be part of KSOR, but KSOR is broader than RAG.

System of context

KSOR content can be connected to real enterprise software/workflows via a "system of context." Vertical KSOR provides domain governance; system of context connects governed rules to real operations.

Hands-on: "Hello World" with KSOR

Prerequisites

- **Install NodeJS**
- Confirm **NodeJS version >= 24**
- Verify with the session's version check command
- If needed, use npm (bundled) or pnpm/bun

Setup steps (high-level)

- Open: **github.com/panover/KSOR**
- Read the "Start here" section in README
- Create a project using the provided command (pnpm example; npm/pnpm/bun options)
- Enter the new project folder (example: "hello KSOR")

- Run commands to test the agent/hello world

Time didn't allow full completion in this class, so participants are asked to complete setup and continue next session.

Q&A highlights

Security for private data (bank/fintech)

- KSOR provides governance boundaries and security responsibilities.
- Deployment security still requires the organization (compliance, secure environment, etc.).

Is deployment knowledge required?

- Yes—you still need to learn how to build agents (technical parts covered elsewhere).
- KSOR focuses on the knowledge layer/governed system of record.
- Deployment engineering details are referenced to the course/tools.

Does attaching a document make the model deterministic?

No—it grounds answers and reduces hallucination within boundaries, but doesn't fully remove stochasticity.

Structured vs unstructured enterprise data

- KSOR adds governed vertical rules/knowledge (e.g., credit analysis rules).
- Vertical KSOR can sit on top of existing systems by pulling needed data through processes.
- System of context connects that governance to real workflows.

Is this "best practice" from big companies?

- They reference broader "system of record" thinking in industry conversations.
- They say they tested similar ideas before teaching.

.